# File Handling Guide

File Handling and Exception Handling

## Python File Handling and Exception Handling – Complete Guide (Basic to Advanced)

File Handling and Exception Handling are two important Python topics for an **AIML Engineer, Data Scientist, and Python Developer**.

You need these concepts to:

- Read datasets such as CSV, TXT, JSON and Excel files.
- Save machine learning model results.
- Store training logs and predictions.
- Handle missing or corrupted files.
- Prevent programs from crashing.
- Build reliable data-processing and ML pipelines.

We will learn both topics step by step with practical examples, code, outputs, dry runs, interview questions, and AIML applications.

## PART 1: FILE HANDLING IN PYTHON

### 1. What is File Handling?

File handling means creating, opening, reading, writing, updating, and deleting files using Python.

**Real-world example:**

Suppose you are building a machine learning model that predicts house prices.

Your project might need to:

1. Read house-price data from `data.csv`.
2. Process the data.
3. Train a model.
4. Save predictions in `predictions.txt`.
5. Save model performance in `results.json`.

Python file handling helps you perform these operations.

## 2. Types of Files

| File type | Purpose | Example |
|---|---|---|
| `.txt` | Plain text | Logs, notes |
| `.csv` | Tabular data | Dataset |
| `.json` | Structured data | Model configuration |
| `.xlsx` | Excel data | Business reports |
| `.jpg`, `.png` | Images | Computer vision |
| `.pkl` | Python objects | ML model storage |

## 3. Opening a File – `open()`

Python provides the built-in `open()` function.

**Syntax:**

```
file = open("filename.txt", "mode")
```

Example:

```
file = open("data.txt", "r")
```

Here:

- `data.txt` = filename.
- `"r"` = reading mode.
- `file` = file object.

## 4. File Opening Modes

This is one of the most important interview topics.

| Mode | Meaning | Description |
|---|---|---|
| `r` | Read | Reads an existing file |
| `w` | Write | Creates or overwrites a file |

| Mode | Meaning | Description |
|---|---|---|
| `a` | Append | Adds content at the end |
| `x` | Create | Creates a new file; fails if it exists |
| `r+` | Read + Write | Reads and writes |
| `w+` | Write + Read | Overwrites and allows reading |
| `a+` | Append + Read | Reads and appends |
| `b` | Binary | Handles binary data |
| `t` | Text | Handles text data (default) |

**Important:** `w` deletes the previous contents of an existing file before writing new content.

## 5. Writing Data into a File

Let's create a file and write machine learning information.

```
file = open("model.txt", "w")

file.write("Machine Learning\n")
file.write("Model Accuracy: 95%\n")
file.write("Algorithm: Random Forest")

file.close()
```

Output inside `model.txt`:

```
Machine Learning
Model Accuracy: 95%
Algorithm: Random Forest
```

**Dry run:**

| Step | Operation | File content |
|---|---|---|
| 1 | Open in `w` mode | Empty |
| 2 | Write Machine Learning | Machine Learning |

| Step | Operation | File content |
|---|---|---|
| 3 | Write accuracy | Adds second line |
| 4 | Write algorithm | Adds third line |
| 5 | Close | Changes saved |

The `write()` function returns the number of characters written.

## 6. Reading Data from a File

Python provides three commonly used methods.

### A. `read()`

Reads the entire file content.

```
file = open("model.txt", "r")

data = file.read()

print(data)

file.close()
```

Output:

```
Machine Learning
Model Accuracy: 95%
Algorithm: Random Forest
```

You can also specify the number of characters:

```
file = open("model.txt", "r")

print(file.read(10))

file.close()
```

Output:

```
Machine Le
```

## B. `readline()`

Reads one line at a time.

```
file = open("model.txt", "r")

print(file.readline())
print(file.readline())

file.close()
```

Output:

```
Machine Learning

Model Accuracy: 95%
```

## C. `readlines()`

Reads all lines and returns them as a list.

```
file = open("model.txt", "r")

data = file.readlines()

print(data)

file.close()
```

Output:

```
[
    'Machine Learning\n',
    'Model Accuracy: 95%\n',
    'Algorithm: Random Forest'
]
```

## Difference between read(), readline(), and readlines()

| Method | Returns | Use |
| --- | --- | --- |
| `read()` | String | Read complete file |

| Method | Returns | Use |
|---|---|---|
| `readline()` | String | Read one line |
| `readlines()` | List | Read all lines |

## 7. Appending Data – `a`

Append means adding new information without deleting the existing content.

```
file = open("model.txt", "a")

file.write("\nTraining Completed")

file.close()
```

New file content:

```
Machine Learning
Model Accuracy: 95%
Algorithm: Random Forest
Training Completed
```

**Real-world use:** Adding new training results to an existing log file.

## 8. Closing a File – `close()`

Always close files after completing operations.

```
file = open("data.txt", "r")

data = file.read()

file.close()
```

Why close files?

- Releases system resources.
- Prevents unnecessary file locks.
- Ensures pending writes are flushed.

## 9. Best Practice – `with open()`

This is the recommended way to work with files in Python.

It automatically closes the file, even if an exception occurs.

```
with open("model.txt", "r") as file:

    data = file.read()

    print(data)
```

You do not need to call `file.close()` manually.

**Interview question: Why is `with open()` better than `open()`?**

Answer: It uses a context manager to automatically manage file resources and close the file when the block exits, including when an exception occurs.

## 10. File Pointer – `tell()` and `seek()`

Python maintains a file pointer that tracks the current reading or writing position.

### A. `tell()`

Returns the current position of the file pointer.

```
with open("model.txt", "r") as file:

    print(file.tell())

    data = file.read(10)

    print(data)
    print(file.tell())
```

Example output:

```
0
Machine Le
10
```

### B. `seek()`

Moves the file pointer to a specified position.

```
with open("model.txt", "r") as file:

    file.seek(0)

    print(file.read(10))
```

Output:

```
Machine Le
```

**Remember:**

- `tell()` = Where am I?
- `seek()` = Move me to a position.

## 11. Reading a File Using a Loop

When processing large datasets or log files, reading line by line is often more memory-efficient than loading the entire file.

```
with open("model.txt", "r") as file:

    for line in file:
        print(line.strip())
```

Output:

```
Machine Learning
Model Accuracy: 95%
Algorithm: Random Forest
```

Here, `strip()` removes unwanted whitespace and newline characters from the beginning and end.

## 12. Creating a New File – `x` Mode

The `x` mode creates a new file.

```
with open("experiment.txt", "x") as file:

    file.write("My first ML experiment")
```

If `experiment.txt` already exists, Python raises:

```
FileExistsError
```

## 13. Binary File Handling – `rb` and `wb`

Binary mode is used for files that contain binary data rather than ordinary text.

Examples:

- Images
- Audio
- Videos
- Serialized Python objects

### Reading an image

```
with open("image.jpg", "rb") as file:

    image_data = file.read()

    print(type(image_data))
```

Output:

```
<class 'bytes'>
```

### Writing binary data

```
with open("copy.jpg", "wb") as file:

    file.write(image_data)
```

This creates a copy of the image.

**AIML application:** Reading image bytes and working with binary files in computer vision applications.

## 14. File and Directory Operations – `os` Module

Python's `os` module provides operations for working with files and folders.

```
import os
```

| Function | Purpose |
|---|---|
| `os.getcwd()` | Current working directory |
| `os.listdir()` | List directory contents |
| `os.mkdir()` | Create directory |
| `os.makedirs()` | Create nested directories |
| `os.remove()` | Delete a file |
| `os.rmdir()` | Delete an empty directory |
| `os.path.exists()` | Check whether a path exists |
| `os.path.isfile()` | Check whether a path is a file |
| `os.path.isdir()` | Check whether a path is a directory |

Example:

```
import os

if os.path.exists("model.txt"):
    print("File exists")
else:
    print("File does not exist")
```

Output:

```
File exists
```

## Create a folder

```
import os

os.makedirs("ML_Project/data", exist_ok=True)
```

This creates the folder structure if it does not already exist.

## 15. Modern Path Handling – `pathlib`

`pathlib` is a modern and convenient way to work with file paths.

```
from pathlib import Path

file_path = Path("data.txt")

print(file_path.exists())
print(file_path.name)
print(file_path.suffix)
```

Output (assuming the file exists):

```
True
data.txt
.txt
```

Creating and writing a file:

```
from pathlib import Path

file_path = Path("results.txt")

file_path.write_text("Model accuracy: 96%")

data = file_path.read_text()

print(data)
```

Output:

```
Model accuracy: 96%
```

## 16. Working with CSV Files

CSV means Comma-Separated Values.

CSV files are commonly used to store datasets in machine learning and data science.

Example `students.csv`:

```
name,age,marks
Aamir,22,85
Rahul,23,90
Priya,21,95
```

Python provides a built-in `csv` module.

## A. Reading CSV

```
import csv

with open("students.csv", "r", newline="") as file:

    reader = csv.reader(file)

    for row in reader:
        print(row)
```

Output:

```
['name', 'age', 'marks']
['Aamir', '22', '85']
['Rahul', '23', '90']
['Priya', '21', '95']
```

## B. Writing CSV

```
import csv

data = [
    ["name", "accuracy", "algorithm"],
    ["Model A", 95, "Random Forest"],
    ["Model B", 92, "Decision Tree"]
]

with open("results.csv", "w", newline="") as file:

    writer = csv.writer(file)

    writer.writerows(data)
```

### C. Reading CSV as dictionaries

```
import csv

with open("students.csv", "r", newline="") as file:

    reader = csv.DictReader(file)

    for row in reader:
        print(row["name"], row["marks"])
```

Output:

```
Aamir 85
Rahul 90
Priya 95
```

**AIML application:** Reading structured training data from CSV files.

## 17. Working with JSON Files

JSON stands for JavaScript Object Notation.

It is commonly used for storing configurations, API responses, and structured model information.

Example `model.json`:

```
{
    "model_name": "Random Forest",
    "accuracy": 0.95,
    "epochs": 100,
    "status": "trained"
}
```

## A. Reading JSON

```
import json

with open("model.json", "r") as file:

    data = json.load(file)

print(data)
print(data["model_name"])
print(data["accuracy"])
```

Output:

```
{'model_name': 'Random Forest', 'accuracy': 0.95, 'epochs': 100, 'status': 'trained'}
Random Forest
0.95
```

## B. Writing JSON

```
import json

model_results = {
    "model_name": "Random Forest",
    "accuracy": 0.95,
    "precision": 0.92,
    "recall": 0.94
}

with open("results.json", "w") as file:

    json.dump(model_results, file, indent=4)
```

`indent=4` makes the JSON file readable.

## C. Difference between `dump()` and `dumps()`

| Function | Purpose |
|---|---|
| `json.dump()` | Writes Python data into a file |
| `json.dumps()` | Converts Python data into a JSON string |

| Function | Purpose |
| --- | --- |
| `json.load()` | Reads JSON from a file |
| `json.loads()` | Converts a JSON string into Python data |

Example:

```
import json

data = {"accuracy": 95}

json_string = json.dumps(data)

print(json_string)
```

Output:

```
{"accuracy": 95}
```

## 18. File Handling in AIML – Practical Project

Let's create a small ML experiment logging system.

**Problem:** You train multiple machine learning models and want to save their accuracy scores.

```
import json

experiments = [
    {
        "model": "Random Forest",
        "accuracy": 0.95
    },
    {
        "model": "Decision Tree",
        "accuracy": 0.89
    },
    {
        "model": "Logistic Regression",
        "accuracy": 0.91
    }
]

# Save experiment results
with open("experiments.json", "w") as file:
    json.dump(experiments, file, indent=4)

# Read experiment results
with open("experiments.json", "r") as file:
    results = json.load(file)

# Find the best model
best_model = max(results, key=lambda x: x["accuracy"])

print("Best Model:", best_model["model"])
print("Accuracy:", best_model["accuracy"])
```

Output:

```
Best Model: Random Forest
Accuracy: 0.95
```

**Concepts used:**

- Lists
- Dictionaries
- File handling
- JSON
- `max()`
- Lambda function

This is a useful example of how Python concepts work together in a real ML workflow.

---

# PART 2: EXCEPTION HANDLING IN PYTHON

## 19. What is Exception Handling?

An exception is an error that occurs while a Python program is running.

Exception handling allows us to manage errors without unnecessarily terminating the program.

**Real-world example:**

Imagine you are building a machine learning application.

Your application tries to read:

```
data = open("dataset.csv", "r")
```

But the file does not exist.

Python raises:

```
FileNotFoundError
```

Without proper handling, the program stops.

With exception handling, you can display a meaningful message and continue safely.

## 20. Error vs Exception

| Error | Exception |
|---|---|
| Broad category of problems | A runtime event that can be handled |
| Includes syntax and runtime problems | Usually occurs during execution |
| Not every error is recoverable | Many exceptions can be handled |

Examples:

```
# Syntax error
if True
    print("Hello")
```

Missing colon causes a `SyntaxError`.

```
# Runtime exception
print(10 / 0)
```

Output:

```
ZeroDivisionError: division by zero
```

## 21. Types of Common Python Exceptions

| Exception | When it occurs |
|---|---|
| `ZeroDivisionError` | Dividing by zero |
| `ValueError` | Correct type but invalid value |
| `TypeError` | Unsupported operation between types |
| `NameError` | Undefined variable |
| `IndexError` | Invalid list index |
| `KeyError` | Missing dictionary key |
| `FileNotFoundError` | File does not exist |
| `PermissionError` | Insufficient file permissions |
| `ImportError` | Import operation fails |
| `ModuleNotFoundError` | Module cannot be found |
| `AttributeError` | Invalid attribute access |
| `TypeError` | Incorrect argument or operation type |
| `EOFError` | Input ends unexpectedly |

| Exception | When it occurs |
|---|---|
| `AssertionError` | Assertion condition is false |
| `JSONDecodeError` | Invalid JSON content |

## 22. `try` and `except`

The `try` block contains code that may cause an exception.

The `except` block handles the exception.

**Syntax:**

```
try:
    # Risky code
    pass

except ExceptionType:
    # Handle error
    pass
```

Example:

```
try:
    result = 10 / 0

except ZeroDivisionError:
    print("Cannot divide by zero")
```

Output:

```
Cannot divide by zero
```

### Dry run

| Step | Operation | Result |
|---|---|---|
| 1 | Enter try block | Start |
| 2 | Execute `10 / 0` | Exception occurs |
| 3 | Skip remaining try statements | Control moves to except |

| Step | Operation | Result |
|---|---|---|
| 4 | Execute except block | Message printed |

## 23. Handling Multiple Exceptions

You can handle different exceptions separately.

```
try:

    num1 = int(input("Enter first number: "))
    num2 = int(input("Enter second number: "))

    result = num1 / num2

    print(result)

except ValueError:
    print("Please enter valid numbers")

except ZeroDivisionError:
    print("Cannot divide by zero")
```

Example 1:

```
Enter first number: abc
Please enter valid numbers
```

Example 2:

```
Enter first number: 10
Enter second number: 0
Cannot divide by zero
```

## 24. Using Multiple Exceptions in One `except`

You can also group multiple exception types.

```
try:

    number = int("abc")

except (ValueError, TypeError):
    print("Invalid input")
```

Output:

```
Invalid input
```

## 25. Using `else` with Exception Handling

The `else` block executes only when no exception occurs in the `try` block.

```
try:

    a = 10
    b = 2

    result = a / b

except ZeroDivisionError:

    print("Division error")

else:

    print("Result:", result)
```

Output:

```
Result: 5.0
```

**Remember:** `else` means no exception occurred.

## 26. Using `finally`

The `finally` block always executes whether an exception occurs or not, under normal control flow.

```
try:
    file = open("data.txt", "r")
    print(file.read())
except FileNotFoundError:
    print("File not found")
finally:
    print("Program execution completed")
```

Output if the file does not exist:

```
File not found
Program execution completed
```

**Real-world use:**

- Closing database connections.
- Releasing resources.
- Cleaning up temporary files.
- Logging completion status.

Remember that `with open()` is generally better for file cleanup.

## 27. Complete Structure – try, except, else, finally

```
try:

    num1 = int(input("Enter first number: "))
    num2 = int(input("Enter second number: "))

    result = num1 / num2

except ValueError:

    print("Invalid input")

except ZeroDivisionError:

    print("Cannot divide by zero")

else:

    print("Result:", result)

finally:

    print("Execution finished")
```

**Execution flow**

**TRY**

Execute risky code

**No exception**

**ELSE**

**Exception**

**EXCEPT**

**FINALLY**

Cleanup / final operation

## 28. Raising Exceptions – `raise`

Sometimes you want to generate an exception yourself when a condition is invalid.

Example:

```
age = -5

if age < 0:
    raise ValueError("Age cannot be negative")
```

Output:

```
ValueError: Age cannot be negative
```

### AIML example: Validate model accuracy

```
accuracy = 1.5

if accuracy < 0 or accuracy > 1:
    raise ValueError("Accuracy must be between 0 and 1")
```

This prevents invalid evaluation values from entering your application.

## 29. Custom Exceptions

You can create your own exception classes.

**Syntax:**

```
class MyError(Exception):
    pass
```

Example:

```
class InvalidAccuracyError(Exception):
    pass

def validate_accuracy(accuracy):

    if accuracy < 0 or accuracy > 1:

        raise InvalidAccuracyError(
            "Accuracy must be between 0 and 1"
        )

    return "Valid accuracy"

try:

    print(validate_accuracy(0.95))

    print(validate_accuracy(1.5))

except InvalidAccuracyError as e:

    print("Error:", e)
```

Output:

```
Valid accuracy
Error: Accuracy must be between 0 and 1
```

**Why use custom exceptions?**

They make applications easier to understand, debug, and maintain.

## 30. Exception Object – `as e`

You can access the exception message using `as`.

```
try:

    result = 10 / 0

except ZeroDivisionError as e:

    print("Error occurred:", e)
```

Output:

```
Error occurred: division by zero
```

You can also print the exception type:

```
try:
    int("hello")

except Exception as e:

    print(type(e).__name__)
    print(e)
```

Output:

```
ValueError
invalid literal for int() with base 10: 'hello'
```

## 31. Exception Hierarchy

Python exceptions follow a class hierarchy.

**BaseException**

**Exception**

Most application errors

- ValueError
- TypeError
- OSError
- RuntimeError
- LookupError

**Other BaseException branches**

Usually not caught by ordinary application handlers

- KeyboardInterrupt
- SystemExit
- GeneratorExit

Important: Most application exceptions inherit from `Exception`. Avoid catching `BaseException` in ordinary application code because it also catches signals such as `KeyboardInterrupt` and `SystemExit`.

---

# PART 3: COMBINING FILE HANDLING AND EXCEPTION HANDLING

This is especially important for AIML engineers because datasets can be missing, corrupted, empty, or incorrectly formatted.

## 32. Practical Project – Safely Read a Dataset

**Problem:** Read a dataset file and handle possible errors.

```
def read_dataset(filename):

    try:

        with open(filename, "r") as file:

            data = file.read()

            if not data.strip():
                raise ValueError("Dataset is empty")

            print("Dataset loaded successfully")
            print(data)

    except FileNotFoundError:

        print("Error: Dataset file not found")

    except PermissionError:

        print("Error: Permission denied")

    except ValueError as e:

        print("Error:", e)

    finally:

        print("Dataset operation completed")

read_dataset("dataset.csv")
```

Possible output if the file does not exist:

```
Error: Dataset file not found
Dataset operation completed
```

## Dry run

| Step | Operation | Explanation |
|---|---|---|
| 1 | Call function | `read_dataset()` |
| 2 | Open file | Attempts to open dataset |
| 3 | File missing | `FileNotFoundError` |
| 4 | Except executes | Displays error |

| Step | Operation | Explanation |
|---|---|---|
| 5 | Finally executes | Displays completion message |

## 33. Practical Project – Save ML Model Results Safely

```
import json

def save_results(filename, results):

    try:

        with open(filename, "w") as file:

            json.dump(results, file, indent=4)

        print("Results saved successfully")

    except PermissionError:

        print("You don't have permission to write this file")

    except OSError as e:

        print("File operation failed:", e)

results = {
    "model": "Random Forest",
    "accuracy": 0.95,
    "precision": 0.92,
    "recall": 0.94
}

save_results("model_results.json", results)
```

Output:

```
Results saved successfully
```

## 34. Practical Project – Dataset Validation

Suppose you receive data from an external source. Before training your model, validate it.

```
import csv

def validate_dataset(filename):

    try:

        with open(filename, "r", newline="") as file:

            reader = csv.DictReader(file)

            required_columns = ["age", "salary", "target"]

            if reader.fieldnames is None:
                raise ValueError("CSV header is missing")

            missing_columns = [
                column
                for column in required_columns
                if column not in reader.fieldnames
            ]

            if missing_columns:
                raise ValueError(
                    f"Missing columns: {missing_columns}"
                )

            print("Dataset validation successful")

    except FileNotFoundError:
        print("Dataset not found")

    except ValueError as e:
        print("Dataset validation failed:", e)

    except csv.Error as e:
        print("CSV processing error:", e)

validate_dataset("training_data.csv")
```

This is an example of a data validation step before the ML training stage.

## 35. Important Advanced Concept – Exception Handling Best Practices

1. Catch specific exceptions instead of catching everything.
2. Avoid empty `except` blocks.
3. Use `with open()` for automatic resource management.
4. Use `raise` to report invalid conditions.

5. Use custom exceptions for application-specific errors.
6. Keep error messages meaningful.
7. Do not hide unexpected errors silently.
8. Use logging instead of relying only on `print()` in production applications.

Example of good practice:

```
try:

    number = int("abc")

except ValueError as e:

    print("Invalid number:", e)
```

Avoid:

```
try:
    number = int("abc")

except:
    pass
```

The second example hides the error and makes debugging difficult.

---

# PART 4: MOST IMPORTANT DIFFERENCES FOR INTERVIEWS

## 36. File Handling vs Exception Handling

| File Handling | Exception Handling |
|---|---|
| Manages files | Manages runtime exceptions |
| Reads and writes data | Handles unexpected situations |
| Uses `open()` | Uses `try` |
| Uses `read()` and `write()` | Uses `except` |
| Uses `close()` | Uses `finally` |

| File Handling | Exception Handling |
|---|---|
| Uses `with open()` | Uses `raise` |

## 37. Important Python Comparisons

| Comparison | Difference |
|---|---|
| `read()` vs `readline()` | Entire content vs one line |
| `readline()` vs `readlines()` | One line vs list of lines |
| `w` vs `a` | Overwrite vs append |
| `r` vs `r+` | Read-only vs read/write |
| `dump()` vs `dumps()` | JSON to file vs JSON string |
| `load()` vs `loads()` | JSON from file vs JSON string |
| `try` vs `except` | Risky code vs error handling |
| `else` vs `finally` | Runs when no exception vs normally always runs |
| `raise` vs `assert` | Explicitly raises an exception vs checks a debugging assumption |

## 38. Important File Handling and Exception Handling Interview Questions

### Beginner-level questions

**1. What is file handling in Python?**

File handling is the process of creating, opening, reading, writing, updating, and deleting files.

**2. What is the purpose of `open()`?**

It opens a file and returns a file object.

**3. What is the difference between `w` and `a` mode?**

`w` overwrites existing content, while `a` adds content at the end.

**4. What does `close()` do?**

It closes a file and releases its associated resources.

**5. What is the advantage of `with open()`?**

It automatically closes the file after the context manager block finishes.

**6. What is an exception?**

An exception is an event that interrupts normal program execution.

**7. What is the purpose of `try` and `except`?**

They allow us to detect and handle exceptions.

**8. What is `finally`?**

It is a block that normally executes regardless of whether an exception occurred.

**9. What is `raise`?**

It explicitly generates an exception.

**10. What is a custom exception?**

A user-defined exception class that usually inherits from `Exception`.

## Intermediate-level questions

**11. What happens if you open a missing file in read mode?**

Python raises `FileNotFoundError`.

**12. What happens when you open an existing file in write mode?**

Its previous contents are truncated.

**13. Can we use multiple except blocks?**

Yes. Different exception types can be handled separately.

**14. Can `finally` execute without `except`?**

Yes.

```
try:
    print("Hello")

finally:
    print("Completed")
```

**15. What is the difference between `json.load()` and `json.loads()`?**

`load()` reads JSON from a file-like object, while `loads()` parses a JSON string.

**16. What is the difference between `read()` and iterating over a file?**

`read()` can load the entire file into memory, whereas iteration reads it line by line.

**17. Why should we avoid a bare `except`?**

It can hide unexpected exceptions and make debugging difficult.

**18. What is `FileNotFoundError`?**

It is an exception raised when an operation attempts to access a file that cannot be found.

**19. What is the difference between syntax errors and runtime exceptions?**

Syntax errors prevent code from being parsed correctly; runtime exceptions occur while executing valid syntax.

**20. What is exception propagation?**

When an exception is not handled in the current function, it travels up the call stack until a suitable handler is found.

## Advanced-level questions

**21. What is a context manager?**

An object that manages setup and cleanup operations using the `with` statement.

**22. What are `__enter__()` and `__exit__()`?**

They are special methods used by the context management protocol.

**23. What is exception chaining?**

It preserves the original exception when raising a new exception using `raise ... from ...`.

```
try:
    int("abc")

except ValueError as e:
    raise RuntimeError("Data conversion failed") from e
```

**24. What is the difference between `Exception` and `BaseException`?**

`BaseException` is the root of Python's built-in exception hierarchy. `Exception` is the parent of most ordinary application exceptions.

### 25. Can we raise an exception inside an except block?

Yes. We can raise a new exception or re-raise the current exception using `raise`.

---

# PART 5: CODING PRACTICE QUESTIONS WITH SOLUTIONS

These problems are useful for Python, Data Science, and AIML interviews.

## Level 1 – Beginner

### Q1. Create a file and write your name.

```
with open("name.txt", "w") as file:

    file.write("Aamir Hussain Azad")

print("File created")
```

### Q2. Read a file and print its content.

```
with open("name.txt", "r") as file:

    content = file.read()

    print(content)
```

### Q3. Count the number of lines in a file.

```
with open("data.txt", "r") as file:

    lines = file.readlines()

    print("Total lines:", len(lines))
```

### Q4. Count words in a file.

```
with open("data.txt", "r") as file:

    content = file.read()

    words = content.split()

    print("Total words:", len(words))
```

### Q5. Handle division by zero.

```
try:

    a = 10
    b = 0

    print(a / b)

except ZeroDivisionError:

    print("Cannot divide by zero")
```

## Level 2 – Intermediate

### Q6. Count characters in a file.

```
with open("data.txt", "r") as file:

    content = file.read()

    print("Characters:", len(content))
```

## Q7. Copy content from one file to another.

```
with open("source.txt", "r") as source:

    content = source.read()

with open("destination.txt", "w") as destination:

    destination.write(content)

print("File copied successfully")
```

## Q8. Find the number of occurrences of a word.

```
with open("data.txt", "r") as file:

    content = file.read()

    count = content.lower().count("python")

    print("Python occurrences:", count)
```

## Q9. Handle multiple exceptions.

```
try:

    numbers = [10, 20, 30]

    index = int(input("Enter index: "))

    print(numbers[index])

except ValueError:
    print("Enter a valid integer")

except IndexError:
    print("Index does not exist")
```

### Q10. Read a JSON file and display model accuracy.

```
import json

try:

    with open("model.json", "r") as file:

        data = json.load(file)

    print("Accuracy:", data["accuracy"])

except FileNotFoundError:
    print("Model file not found")

except json.JSONDecodeError:
    print("Invalid JSON file")

except KeyError:
    print("Accuracy field is missing")
```

## Level 3 – AIML Engineer Coding Questions

### Q11. Save multiple model accuracies into a file.

```
import json

models = {
    "Random Forest": 0.95,
    "Decision Tree": 0.89,
    "Logistic Regression": 0.91,
    "SVM": 0.93
}

with open("accuracy.json", "w") as file:

    json.dump(models, file, indent=4)

print("Model results saved")
```

## Q12. Find the best-performing model from JSON data.

```
import json

with open("accuracy.json", "r") as file:

    models = json.load(file)

best_model = max(models, key=models.get)

print("Best model:", best_model)
print("Accuracy:", models[best_model])
```

Expected output:

```
Best model: Random Forest
Accuracy: 0.95
```

## Q13. Validate model accuracy using exception handling.

```
def validate_accuracy(accuracy):

    try:

        accuracy = float(accuracy)

        if not 0 <= accuracy <= 1:
            raise ValueError("Accuracy must be between 0 and 1")

        return "Valid accuracy"

    except (ValueError, TypeError) as e:

        return f"Invalid accuracy: {e}"

print(validate_accuracy(0.95))
print(validate_accuracy(1.5))
print(validate_accuracy("hello"))
```

Output:

```
Valid accuracy
Invalid accuracy: Accuracy must be between 0 and 1
Invalid accuracy: could not convert string to float: 'hello'
```

## Q14. Create a training log file.

```
from datetime import datetime

def save_training_log(model, accuracy):

    timestamp = datetime.now()

    with open("training_log.txt", "a") as file:

        file.write(f"Time: {timestamp}\n")
        file.write(f"Model: {model}\n")
        file.write(f"Accuracy: {accuracy}\n")
        file.write("-" * 30 + "\n")

save_training_log("Random Forest", 0.95)
save_training_log("SVM", 0.93)

print("Training logs saved")
```

This is useful for tracking multiple model experiments.

## Q15. Build a complete safe file-reading function.

```
import json

def load_model_results(filename):

    try:

        with open(filename, "r") as file:

            results = json.load(file)

        if not isinstance(results, dict):
            raise ValueError("Expected a JSON object")

        if "accuracy" not in results:
            raise KeyError("Accuracy field missing")

        accuracy = float(results["accuracy"])

        if not 0 <= accuracy <= 1:
            raise ValueError("Invalid accuracy range")

        return results

    except FileNotFoundError:
        print("File not found")

    except json.JSONDecodeError:
        print("Invalid JSON format")

    except (ValueError, TypeError, KeyError) as e:
        print("Invalid model results:", e)

    return None

result = load_model_results("model_results.json")

if result is not None:
    print(result)
```

This combines file handling, JSON, functions, dictionaries, validation, and exception handling.

# PART 6: REVISION NOTES – MEMORIZE BEFORE INTERVIEW

## 39. File Handling Quick Revision

### Important functions

| Function | Remember |
|---|---|
| `open()` | Open file |
| `read()` | Read all content |
| `readline()` | Read one line |
| `readlines()` | Read all lines as list |
| `write()` | Write string |
| `writelines()` | Write multiple strings |
| `close()` | Close file |
| `seek()` | Move file pointer |
| `tell()` | Get pointer position |
| `json.load()` | Read JSON file |
| `json.dump()` | Write JSON file |

### File modes

- `r` = Read
- `w` = Write / overwrite
- `a` = Append
- `x` = Create
- `b` = Binary
- `t` = Text
- `r+` = Read and write

## 40. Exception Handling Quick Revision

### Five important keywords

| Keyword | Meaning |
|---|---|
| `try` | Risky code |
| `except` | Handle exception |
| `else` | Runs when no exception occurs |
| `finally` | Cleanup code |
| `raise` | Generate exception |

### Important exceptions to memorize

- `ValueError`
- `TypeError`
- `ZeroDivisionError`
- `IndexError`
- `KeyError`
- `NameError`
- `FileNotFoundError`
- `PermissionError`
- `AttributeError`
- `JSONDecodeError`

## 41. Practice Test – Check Your Knowledge

Try answering these without looking at the notes.

**Python interview mini-test** (/10 answered)

## 42. Your AIML Engineer Practice Checklist

### Learning progress

### %

of completed

**My recommendation:** Learn File Handling first, then Exception Handling, and finally combine both in a dataset-loading project. This gives you practical knowledge rather than just memorizing definitions.

I can also prepare a 20-question Python File Handling and Exception Handling coding interview exercise with solutions and dry runs for tomorrow's practice.

Daily Python coding practice